

AWS + Snowflake for Data Engineers & Analysts

Fundamentals, Snowflake explained for good, and two complete hands-on projects — built to fit inside AWS's free-tier limits.

Fundamentals

The core vocabulary every data engineer and analyst needs — pipelines, ETL vs ELT, modeling, OLTP vs OLAP.

AWS, the Right Slice

The AWS services that actually matter for data work, what they cost, and what's genuinely free.

Snowflake, for Good

One analogy that makes the whole architecture click — plus every feature you'll actually use.

Two hands-on projects included · Both fit inside AWS Free Tier limits · Every console step spelled out

Prepared for Aayush — B.Tech CSE (Data Science) · Data Engineering Portfolio & Interview-Readiness Track

July 2026

Contents

1. Data Engineering & Analytics — The Fundamentals	3
2. AWS Fundamentals — The Services That Make You Industry-Ready	5
2.1 Identity & Security — the layer everything else sits on	5
2.2 Storage — the foundation of your data lake	5
2.3 Compute	5
2.4 Networking — need-to-know, not need-to-master	6
2.5 Databases	6
2.6 The Analytics Stack — where you'll actually live	7
2.7 Orchestration & Monitoring	7
2.8 Serving & BI	7
3. Snowflake — Explained So You Never Forget It	9
3.1 The big idea: storage and compute live apart	9
3.2 Every feature you'll actually use — with a hook that sticks	10
4. Project 1 — The Habit-BUILDER (30–45 minutes)	11
5. Capstone — Serverless E-Commerce Analytics Pipeline	13
6. What's Next — Bringing AWS and Snowflake Together	18
Golden rules — never get a surprise AWS bill	18

1. Data Engineering & Analytics — The Fundamentals

THE SHARED FOUNDATION BEHIND EVERYTHING ELSE IN THIS DOCUMENT

Two roles, one goal: turning raw data into decisions someone can act on. Everything in the AWS and Snowflake sections that follow is just tooling in service of the ideas below — learn these once, and every new tool becomes easy to place.

The two roles, in one line each

- **Data Engineer** — builds and maintains the pipes: systems that reliably move, store, and reshape data at scale. Think plumbing, not the water itself.
- **Data Analyst** — turns stored data into answers: trends, KPIs, forecasts — delivered through dashboards and clear stakeholder communication.
- Both roles live or die by one shared skill: **SQL**. Everything else is a specialization on top of it.

Anatomy of a pipeline — six stages, every single time

The Data Pipeline, Start to Finish



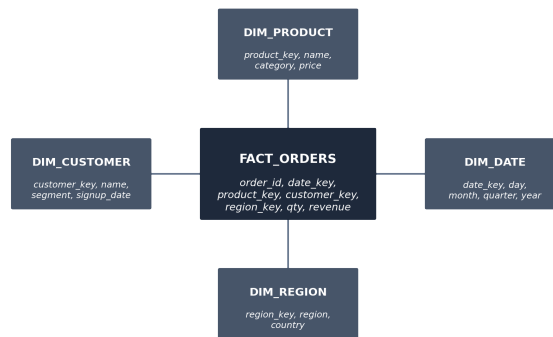
Vocabulary you must have cold

Term	What it means	Real example
ETL	Transform <i>before</i> loading — the classic pattern from when compute was expensive.	A nightly script cleans data, then loads it into the warehouse.
ELT	Load raw data first, transform <i>inside</i> the warehouse using its elastic compute.	Snowflake ingests raw JSON; a SQL/dbt model reshapes it — today's default.
OLTP	Transactional app database — many small, fast reads/writes, normalized.	The Postgres database behind a checkout button.
OLAP	Analytical warehouse — few large scans across millions of rows, denormalized.	"Total revenue by region, last quarter" — run in Snowflake or Redshift.

Modeling: the star schema

One **fact table** (what happened — mostly numbers and foreign keys) surrounded by **dimension tables** (who / what / when / where). This is exactly the shape you already built in your PTU and TPCH Snowflake projects — it's the industry-standard layout for any BI-friendly warehouse.

Star Schema — One Fact Table, Many Dimension Tables



Batch vs. streaming, and staying honest about quality

- **Batch**: process data in scheduled chunks (nightly/hourly) — simple, cheap, fine for most reporting.
- **Streaming**: process each event the moment it lands (Kafka, Kinesis, Snowpipe) — needed for fraud alerts and live dashboards.
- **Orchestration** (Airflow, Step Functions, dbt) schedules multi-step jobs, retries failures, and gives you one place to check whether last night's run actually succeeded.
- **Data quality** — nulls, duplicates, schema drift — isn't optional. A dashboard is only as trustworthy as the pipeline feeding it.

TIP — Data Engineer builds the road. Data Analyst drives it well — and tells everyone where it leads.

2. AWS Fundamentals — The Services That Make You Industry-Ready

AWS HAS 200+ SERVICES — YOU NEED ABOUT 15 OF THEM, DEEPLY

This section is organized the way a real pipeline is organized — security first, then storage, compute, databases, the analytics stack, orchestration, and serving. Everything here is a service you'll actually touch in the projects that follow.

AWS NOTE — AWS overhauled its Free Tier on **July 15, 2025**. Accounts created after that date choose a **Free Plan** or **Paid Plan** at signup — both start with \$100 in credit (up to \$200 after 5 onboarding tasks). The Free Plan auto-closes after 6 months or when credits run out, whichever comes first. Over 30 **Always Free** services (Lambda, DynamoDB, Glue Catalog, CloudWatch...) have permanent monthly limits that never expire, on either plan. Accounts created *before* July 15, 2025 keep the old 12-month free tier on EC2/RDS/S3. Practical advice: pick the **Paid Plan** even as a learner and set a billing alarm — you're only ever charged for what you use beyond the always-free limits, and both projects in this document cost a few cents, total.

2.1 Identity & Security — the layer everything else sits on

- **IAM (Identity & Access Management)**: users, groups, roles, and JSON policies that define exactly who can do what. Free, always.
- **Users vs. Roles**: a user is a permanent identity with long-lived credentials. A **role** is a temporary hat — any trusted identity (a person, a Lambda function, a Glue job) can put it on, use it, and take it off. In real pipelines, services almost always talk to each other via roles, never hardcoded keys.
- **Principle of least privilege**: grant only the exact permissions a task needs — nothing 'just in case.'

2.2 Storage — the foundation of your data lake

- **Amazon S3**: object storage — buckets hold objects, addressed by key. This is where 90% of a data engineer's raw and processed data physically lives.
- **Storage classes**: Standard (frequent access) → Standard-IA (infrequent) → Glacier (archive, cheapest, slow retrieval). **Lifecycle rules** auto-move objects between classes as they age.
- **Versioning** keeps every past copy of an object — your safety net against accidental overwrites.

COST CHECK — **Always Free**: 5 GB Standard storage, 20,000 GET and 2,000 PUT requests per month, forever.

2.3 Compute

- **EC2**: virtual servers. Instance families signal purpose — **t** = burstable general purpose, **c** = compute-optimized, **r** = memory-optimized. Pricing: On-Demand (flexible), Reserved (cheaper, 1–3 yr commitment), Spot (cheapest, can be reclaimed anytime).
- **Lambda**: serverless functions — code that runs only when triggered (a file lands in S3, a schedule fires) and scales to zero when idle. This is your best friend for lightweight automation.

COST CHECK — **Legacy 12-month free** (pre-July-2025 accounts only): EC2 — 750 hrs/month of t2.micro or t3.micro. **Always Free** (any account, forever): Lambda — 1 million requests + 400,000 GB-seconds of compute per month.

2.4 Networking — need-to-know, not need-to-master

- **VPC**: your own private network inside AWS. **Subnets** are public (internet-facing) or private (internal only). **Security groups** act as a virtual firewall per resource.
- As a data engineer/analyst you rarely configure this deeply — but you should recognize the vocabulary when a platform team mentions it.

2.5 Databases

- **RDS**: managed relational databases (Postgres, MySQL, MariaDB) — AWS handles patching and backups for you. Legacy 12-month free: 750 hrs/month db.t2.micro/db.t3.micro (pre-July-2025 accounts). Aurora PostgreSQL Serverless joined the Always Free tier in March 2026.
- **DynamoDB**: managed NoSQL, key-value/document store, single-digit-millisecond latency at any scale. Always Free: 25 GB storage and a generous monthly request allowance, forever.

2.6 The Analytics Stack — where you'll actually live

- **AWS Glue — Data Catalog:** a persistent, central metastore of table schemas — think of it as the card catalog for your entire data lake. Athena, Redshift Spectrum, and EMR all read from it. Always Free: 1 million objects stored + 1 million requests per month.
- **AWS Glue — Crawlers:** point one at an S3 folder and it automatically infers schema and populates the Data Catalog. Billed per DPU-hour, but a few cents at most for small data.
- **AWS Glue — ETL Jobs:** serverless Spark (PySpark) or Python-shell scripts for transformation — no clusters to provision. ~\$0.44 per DPU-hour, billed by the second with a short minimum, so a learning-sized job costs a few cents per run.
- **Amazon Athena:** serverless SQL directly over data sitting in S3 — zero infrastructure. \$5 per TB scanned, with a 10 MB minimum charge per query (about \$0.00005 at minimum). **Important:** on new (post-July-2025) *Free Plan* accounts, Athena isn't enabled — you need the Paid Plan to use it. Cost stays negligible at the data sizes in this document.
- **Amazon Redshift:** AWS's dedicated cloud data warehouse — Snowflake's most direct competitor. Historically a fixed cluster you pay for even when idle; Redshift Serverless narrows that gap. New users get a separate \$300 / 90-day Redshift Serverless trial (not part of the AWS Free Tier) — good to know it exists, but we'll use Athena in these projects to stay reliably free.
- **EMR:** managed Hadoop/Spark clusters for genuinely big data. No meaningful free tier — worth recognizing by name, but skip it for hands-on practice to avoid cost risk.
- **Kinesis:** real-time streaming ingestion (Data Streams + Firehose) — AWS's answer to Kafka, for when data must be processed the instant it arrives.

2.7 Orchestration & Monitoring

- **Step Functions:** a visual state-machine that chains Lambda/Glue/other steps together with retries and branching logic. Always Free: 4,000 state transitions per month.
- **EventBridge:** an event bus — “when a file lands in S3, trigger this Lambda” or “run this every night at 2 AM.” The glue between services.
- **CloudWatch:** logs, metrics, and alarms. Non-negotiable: set a **billing alarm** here on day one of any AWS account. Always Free: 10 custom metrics, 10 alarms, 1 million API requests, 5 GB log ingestion per month.

2.8 Serving & BI

- **QuickSight:** AWS's native BI/dashboard tool. 30-day free trial (Standard edition, 1 author), then paid. Great for a demo; for a permanently-free portfolio piece, export Athena results and chart them with Python instead — exactly what Project 2 does.

Quick-reference: the AWS cheat sheet

Service	Category	Role in your pipeline	Free-tier snapshot
IAM	Security	Grants least-privilege access to every other service	Always free
S3	Storage	Home for raw & processed data (the data lake)	5 GB, 20k GET, 2k PUT / mo
Lambda	Compute	Event-driven automation & light transforms	1M reqs + 400k GB-sec / mo

Service	Category	Role in your pipeline	Free-tier snapshot
EC2	Compute	General-purpose virtual servers	Legacy: 750 hrs/mo (pre-Jul-2025 accts)
RDS	Database	Managed OLTP database (Postgres/MySQL)	Legacy: 750 hrs/mo (pre-Jul-2025 accts)
DynamoDB	Database	Managed NoSQL at any scale	25 GB + high request volume, always
Glue Catalog	Analytics	Central metadata store for the data lake	1M objects + 1M requests / mo
Glue Crawler	Analytics	Auto-detects schema from raw files	Pay per DPU-hr ($\approx$ cents)
Glue ETL Job	Analytics	Serverless PySpark transformation	Pay per DPU-hr ($\approx$ cents)
Athena	Analytics	Serverless SQL directly over S3	No free tier; \$5/TB, 10MB min/query
Redshift	Analytics	Dedicated cloud data warehouse	\$300/90-day Serverless trial (separate)
Step Functions	Orchestration	Visual multi-step workflow orchestrator	4,000 transitions / mo
EventBridge	Orchestration	Event bus — triggers & schedules	Generous always-free tier
CloudWatch	Monitoring	Logs, metrics, alarms — incl. billing alarms	10 metrics/alarms, 5GB logs / mo
QuickSight	BI / Serving	Native AWS dashboards	30-day trial, then paid

3. Snowflake — Explained So You Never Forget It

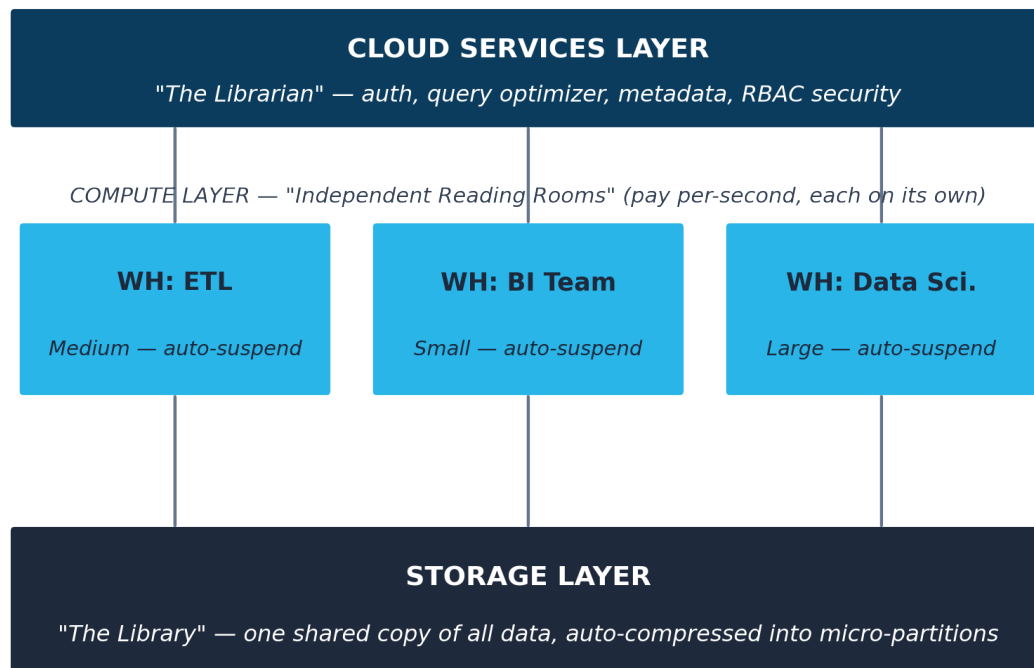
ONE ANALOGY UNLOCKS THE WHOLE ARCHITECTURE

Picture a library where the **librarian**, the **books**, and the **reading rooms** are three completely separate things that just happen to work together perfectly. That's Snowflake. Hold onto that picture — every feature below is just a consequence of it.

3.1 The big idea: storage and compute live apart

Traditional databases bundle storage and compute together — like a library where the books live *inside* each reading room. Add more readers and you end up duplicating the books. Snowflake instead splits everything into three independent layers:

Snowflake Architecture — Storage and Compute Live Apart



- **Storage layer — “The Library.”** One shared, compressed copy of all your data, automatically sliced into **micro-partitions** (50–500 MB chunks with min/max metadata) — like pre-labeled shelves, so a query knows exactly which shelf to check instead of scanning the whole library. You never run `CREATE INDEX`; this happens by default on load.
- **Compute layer — “Independent Reading Rooms.”** Virtual Warehouses, any number, any size (T-shirt sizes XS through 6XL, each size = double the credits/hour of the one below). Every warehouse can read the *same* library at the same time without ever bumping into another. **Auto-suspend** turns the lights off (and billing off) the second a room empties out; **auto-resume** turns them back on in seconds. You pay per-second, only while a warehouse runs.

- **Cloud Services layer — “The Librarian.”** Invisible to you — it authenticates every request, parses and optimizes every query, tracks all metadata, and enforces role-based security. You never provision it and never pay for it directly.

SNOWFLAKE NOTE — Because compute and storage scale *independently*, you can run a heavy ETL job on a big warehouse for ten minutes and a dashboard on a tiny warehouse all day — and pay for each separately, with zero contention. Traditional systems (and older Redshift clusters) couple the two, so teams over-provision “just in case.” Fair-comparison note: Redshift Serverless has narrowed this gap in recent years.

3.2 Every feature you'll actually use — with a hook that sticks

Feature	The explanation that sticks
Micro-partitions	Auto-labeled shelves (min/max stats) so queries skip irrelevant data. No manual indexing, ever — it happens automatically on load.
Time Travel	"Ctrl+Z for your data." Query or restore a table exactly as it looked minutes or days ago: <code>SELECT * FROM orders AT(OFFSET => -3600);</code> OR <code>UNDROP TABLE orders;</code>
Zero-Copy Cloning	"An Instagram repost, not a download." <code>CREATE TABLE dev CLONE orders;</code> gives you a full working copy in about a second — metadata-only, costs nothing extra until the clone's data actually diverges.
Streams	A sticky note that remembers exactly which rows changed (insert/update/delete) since you last looked — the engine behind Change Data Capture (CDC).
Tasks	A scheduled alarm clock: wakes up, runs one SQL statement (often “process whatever’s on the Stream”), goes back to sleep. Streams + Tasks = a serverless ELT pipeline, zero orchestration tool required.
Snowpipe	The mail slot that processes each letter (file) the instant it lands in cloud storage — continuous, event-driven loading, no manual <code>COPY INTO</code> needed.
Semi-structured data	Drop a whole JSON document into one <code>VARIANT</code> column and query nested fields with dot notation — no separate NoSQL store needed for messy data.
Roles & RBAC	Access is 100% role-based, never user-based directly. A user can hold many roles (hats) and switch which one they're wearing with <code>USE ROLE</code> — exactly the pattern you hit on your PTU project's free-trial RBAC limits.
Secure Data Sharing	"A library card to your library, not a photocopy of your books." Another Snowflake account gets live, real-time read access to your data — zero copying, zero ETL.

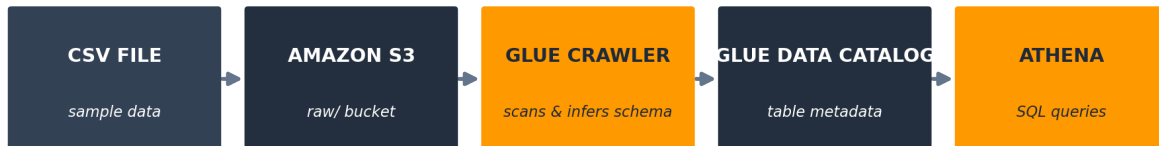
TIP — One line to remember forever: **your data has one home, but as many front doors (warehouses) as you want to build — and you only pay rent on the doors you're actually using.**

4. Project 1 — The Habit-Builder (30–45 minutes)

GOAL: BURN THE S3 → GLUE → ATHENA LOOP INTO MUSCLE MEMORY

This is the single most common motion in AWS data work: land a file in S3, catalog it with Glue, query it with Athena. Do this once end-to-end and every future AWS data task feels familiar. Dataset: **starter_sample_sales.csv** (60 rows, provided alongside this document).

Project 1 — The Habit-Builder Loop



Step 0 — Set a billing alarm (2 minutes, do this first, always)

- Console → Billing and Cost Management → Budgets → Create budget → choose a “Zero spend budget” template → confirm. This alerts you the moment you're forecast to spend anything beyond the always-free limits.

Step 1 — Create an S3 bucket

- S3 console → Create bucket → name it something globally unique, e.g. yourname-de-starter-2026 → pick one region and stick with it for the whole project (e.g. **ap-south-1 / Mumbai**, closest to you) → Create.

Step 2 — Upload the CSV

- Inside the bucket, create a folder named `raw/` → upload `starter_sample_sales.csv` into it.

Step 3 — Create an IAM role for Glue

- IAM console → Roles → Create role → Trusted entity: **AWS service** → **Glue** → attach the managed policy `AWSGlueServiceRole` plus S3 read access scoped to your bucket → name it `GlueCrawlerRole`.

Step 4 — Create & run a Glue Crawler

- Glue console → Crawlers → Create crawler → name `sales_crawler` → data source: S3, path `s3://your-bucket/raw/` → IAM role: `GlueCrawlerRole` → target database: create new `sales_db` → Run.
- After 1–2 minutes, check Glue → Tables: a new table appears with columns auto-detected straight from your CSV header row — no schema was typed by hand.

Step 5 — Query with Athena

- Athena console → first-time setup: Settings → set a query result location, e.g. `s3://your-bucket/athena-results/`.
- Choose database `sales_db` and run your first query.

Query 1 — sanity check

```
SELECT * FROM raw LIMIT 10;
```

Step 6 — Three queries to build real muscle memory

Query 2 — total revenue

```
-- total revenue across all orders
SELECT SUM(quantity * unit_price) AS total_revenue FROM raw;
```

Query 3 — revenue by category

```
-- revenue by category, highest first
SELECT category, SUM(quantity * unit_price) AS revenue
FROM raw
GROUP BY category
ORDER BY revenue DESC;
```

Query 4 — best-seller

```
-- best-selling product by units sold
SELECT product_name, SUM(quantity) AS units_sold
FROM raw
GROUP BY product_name
ORDER BY units_sold DESC
LIMIT 1;
```

COST CHECK — Realistic total cost for this entire exercise: a fraction of a cent — a few KB of S3 storage, one short crawl, a handful of tiny Athena queries. Nothing urgent to clean up, but for a zero-footprint account: delete the Athena results folder, drop the table, then delete the bucket.

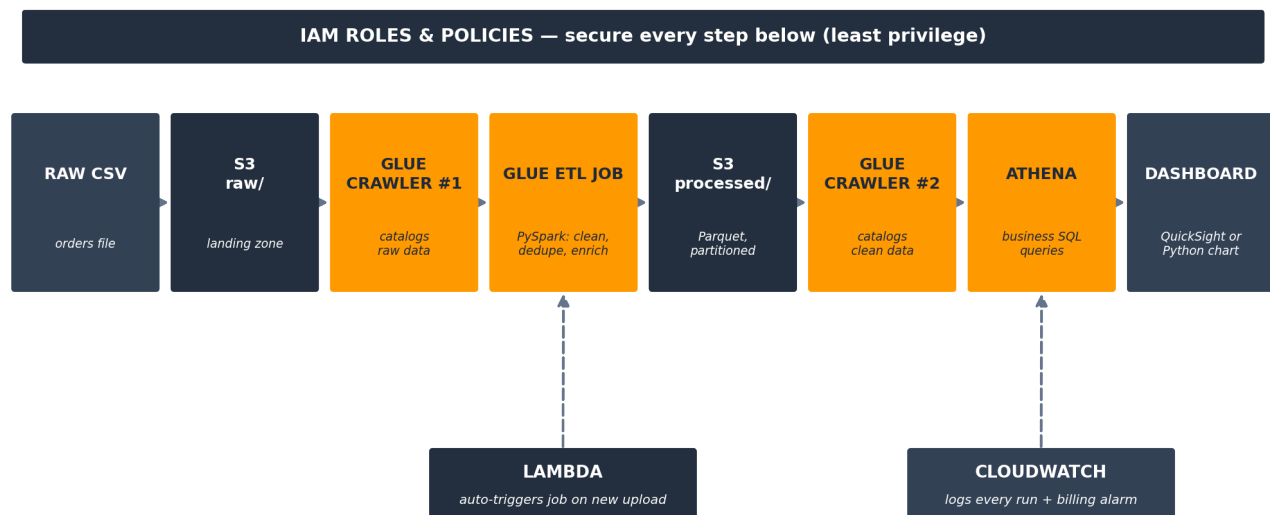
TIP — Habit to build: every AWS data task starts with the same two questions — “*where does the file land, and who is allowed to read it?*” That’s S3 + IAM, always, before anything else.

5. Capstone — Serverless E-Commerce Analytics Pipeline

DATA ENGINEER + DATA ANALYST, COMBINED, STILL FREE-TIER

This is the portfolio piece: a real ELT pipeline that ingests messy raw orders, cleans them with a PySpark job, and serves a partitioned analytics table for business questions. Dataset: **capstone_ecommerce_orders.csv** (435 rows, provided — deliberately messy: duplicates, missing prices, inconsistent category casing, just like real source data). Budget 2–3 sessions.

Project 2 — Capstone Pipeline Architecture



Step 1 — Bucket structure

- Create bucket `yourname-de-capstone-2026` with four folders: `raw/`, `processed/`, `scripts/`, `athena-results/`.
- Upload `capstone_ecommerce_orders.csv` into `raw/`.

Step 2 — IAM role for Glue

- Same as Project 1, plus attach `CloudWatchLogsFullAccess` so job runs write logs you can actually read. Name it `GlueETLRole`.

Step 3 — Glue Crawler #1: catalog the raw data

- Crawler `raw_orders_crawler` on `s3://your-bucket/raw/` → database `ecommerce_db` → produces table `raw_orders`.

Step 4 — Write the Glue ETL Job (the actual engineering)

Glue console → ETL jobs → Create job → Script editor, Glue 4.0, Python → paste the script below. Each numbered comment is a real data-quality decision — in an interview, being able to explain *why* each step exists matters more than the code itself:

glue_etl_job.py

```

import sys
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.job import Job
from pyspark.sql import functions as F

args = getResolvedOptions(sys.argv, ['JOB_NAME'])
sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session
job = Job(glueContext)
job.init(args['JOB_NAME'], args)

# 1. Read the raw table straight from the Glue Data Catalog
raw_dyf = glueContext.create_dynamic_frame.from_catalog(
    database="ecommerce_db", table_name="raw_orders")
df = raw_dyf.toDF()

# 2. Drop exact duplicate rows -- real-world data always has a few
df = df.dropDuplicates()

# 3. Standardize category casing: "ELECTRONICS"/"electronics" -> "Electronics"
df = df.withColumn("category", F.initcap(F.trim(F.col("category"))))

# 4. Drop rows where region is missing -- can't be safely guessed
df = df.filter(F.col("region").isNotNull() & (F.trim(F.col("region")) != ""))

# 5. Fill missing unit_price with the average price for that same product
df = df.withColumn("unit_price", F.col("unit_price").cast("double"))
avg_price = df.groupBy("product_name").agg(F.avg("unit_price").alias("avg_price"))
df = df.join(avg_price, on="product_name", how="left")
df = df.withColumn("unit_price",
    F.coalesce(F.col("unit_price"), F.col("avg_price"))).drop("avg_price")

# 6. Add the computed column that makes this an analytics table
df = df.withColumn("quantity", F.col("quantity").cast("int"))
df = df.withColumn("revenue", F.round(F.col("quantity") * F.col("unit_price"), 2))

# 7. Add year / month so the output can be partitioned
df = df.withColumn("order_date", F.to_date("order_date"))
df = df.withColumn("year", F.year("order_date"))
df = df.withColumn("month", F.month("order_date"))

# 8. Write clean, partitioned Parquet back to S3
df.write.mode("overwrite").partitionBy("year", "month") \
    .parquet("s3://your-bucket/processed/")

job.commit()

```

- Steps 2–4 are **data quality** fixes: exact duplicates removed, casing standardized, unusable rows (no region) dropped.
- Step 5 is a judgment call: rather than dropping rows with a missing price, it's **imputed** using that product's average price — a defensible, explainable choice you should be ready to justify out loud.
- Steps 6–8 turn cleaned data into an **analytics-ready** table: a computed revenue column, and a partition scheme (year/month) so Athena scans only the months a query actually needs.

Step 5 — Run the job & verify

- Run job → wait for status **Succeeded** → browse `s3://your-bucket/processed/` → confirm Parquet files exist inside folders like `year=2026/month=1/`.

Step 6 — Glue Crawler #2: catalog the clean data

- Crawler `processed_orders_crawler` on `s3://your-bucket/processed/` → produces table `processed_orders` in the same database.

Step 7 — Athena: the data-analyst layer

Five business questions, five queries — this is the payoff for all the cleaning above.

Query 1 — monthly revenue trend

```
-- monthly revenue trend
SELECT year, month, ROUND(SUM(revenue), 2) AS monthly_revenue
FROM processed_orders
GROUP BY year, month
ORDER BY year, month;
```

Query 2 — top 5 products

```
-- top 5 products by revenue
SELECT product_name, ROUND(SUM(revenue), 2) AS revenue
FROM processed_orders
GROUP BY product_name
ORDER BY revenue DESC
LIMIT 5;
```

Query 3 — revenue by region

```
-- revenue by region
SELECT region, ROUND(SUM(revenue), 2) AS revenue
FROM processed_orders
GROUP BY region
ORDER BY revenue DESC;
```

Query 4 — average order value

```
-- average order value
SELECT ROUND(AVG(revenue), 2) AS avg_order_value
FROM processed_orders;
```

Query 5 — volume vs. value by category

```
-- category with the most orders but the lowest average order value
-- (a real insight-style question, not just a total)
SELECT category,
       COUNT(*) AS order_count,
       ROUND(AVG(revenue), 2) AS avg_order_value
FROM processed_orders
GROUP BY category
ORDER BY order_count DESC;
```

Step 8 (stretch) — Automate it

Add an S3 event notification (or an EventBridge rule) on raw/ that invokes a small Lambda, which starts the Glue job automatically the moment a new file lands:

trigger_etl.py — Lambda function

```
import boto3

glue = boto3.client("glue")

def lambda_handler(event, context):
    # Triggered by an S3 "ObjectCreated" event notification on raw/
    response = glue.start_job_run(JobName="ecommerce_etl_job")
    print("Started Glue job run:", response["JobRunId"])
    return {"status": "started", "run_id": response["JobRunId"]}
```

Step 9 (optional) — Visualize

- **Path A:** QuickSight 30-day trial → connect to Athena → build 2–3 charts → cancel/downgrade before day 30.
- **Path B (recommended, permanently free):** download each Athena result as CSV → open in a Python notebook → 2–3 pandas/matplotlib charts. Zero risk of forgetting to cancel a trial.

COST CHECK — Realistic total cost: S3 storage (pennies), 2 crawler runs (≈\$0.02–\$0.05 combined), 1–3 Glue ETL runs (≈\$0.02–\$0.05 each), a dozen-plus tiny Athena queries (≈\$0.0005 total). The whole capstone lands comfortably under \$1 — usually a handful of cents — covered many times over by your signup credit.

Step 10 — Cleanup (stop paying for anything, ever)

- Delete the Glue job, both crawlers, both tables, and the database. Empty and delete both buckets. Remove IAM roles if they're not used elsewhere.

INTERVIEW PITCH — Built a serverless ELT pipeline on AWS (S3, Glue, Athena) that ingests raw e-commerce orders, cleans and deduplicates them with a PySpark job, and serves partitioned Parquet tables for sub-second SQL analytics — automated end-to-end with Lambda and monitored via CloudWatch, entirely within AWS free-tier limits.

6. What's Next — Bringing AWS and Snowflake Together

THE NATURAL FOLLOW-ON PROJECT

The cleanest way to make this whole document click into one story: take the exact Parquet output from the capstone's processed/ folder and load it into Snowflake through an external stage, then re-run the same five business questions there. Same data, two engines — you'll feel the storage/compute separation from Section 3 directly, and it plugs straight into the PTU and TPC-H Snowflake projects you've already built.

bridge.sql — AWS S3 → Snowflake

```
-- 1. Point Snowflake at the same S3 folder your Glue job already wrote to
CREATE OR REPLACE STAGE ecommerce_stage
  URL = 's3://your-bucket/processed/'
  CREDENTIALS = (AWS_KEY_ID='...' AWS_SECRET_KEY='...');

-- 2. Load it into a Snowflake table -- same data, new engine
COPY INTO processed_orders
  FROM @ecommerce_stage
  FILE_FORMAT = (TYPE = PARQUET)
  MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE;

-- 3. Re-run Query 1 from the capstone, unchanged, and compare
SELECT year, month, ROUND(SUM(revenue), 2) AS monthly_revenue
FROM processed_orders
GROUP BY year, month
ORDER BY year, month;
```

Golden rules — never get a surprise AWS bill

- **Billing alarm on day one**, every single AWS account, no exceptions.
- **Delete unused resources the day you finish testing** — idle NAT Gateways (~\$33/month just for existing), unattached Elastic IPs, and orphaned EBS volumes are the classic silent bill inflators.
- **Prefer Always Free services** (Lambda, DynamoDB, Glue Catalog) over 12-month-limited ones (EC2, RDS) when you're just prototyping.
- If you're on the new **Free Plan**, remember it auto-closes after 6 months regardless of remaining credit — don't park a long-term project there.
- Check **Cost Explorer weekly** during active project work, not just when the bill lands.

That's the full loop: the vocabulary in Section 1 explains *why* every tool in Sections 2–3 exists, and Sections 4–5 turn that vocabulary into two pipelines you actually built with your own hands, on your own AWS account, for a few cents. Ship both, write them up the way you did BrilliantDex and your Snowflake projects, and you have a genuinely complete data engineering portfolio story to tell in an interview.